

East West University

Department of Computer Science and Engineering

CSE400A Capstone Project Report

PhishGuard

**A Multi-Model Phishing Detection System with Anomaly-Based
Intelligence for Web and Text Threats**

Supervised By:

Dr. Sarwar Jahan

Associate Professor

Department of Computer
Science & Engineering

Submitted By:

Jannatul Ferdous Oishi

2022-3-60-216

Maruf Ahmed Anik

2022-3-60-040

Arham Jawad Akib

2022-3-60-004

Mahin Ahmed Meghla

2022-3-60-152

Submission Date: 19 May, 2026

1. Project Title

PhishGuard: A Multi-Modal Phishing Detection System with Anomaly-Based Intelligence.

The title is chosen to clearly state the subject and scope of the study. PhishGuard refers to the proposed lightweight phishing detection tool. The phrase “multi-modal” indicates that the system analyzes more than one type of input in its core implementation: URLs and text extracted from screenshots. “Anomaly-Based Intelligence” reflects the use of unsupervised machine learning (Isolation Forest) to detect new and previously unseen phishing patterns.

2. Background

2.1 The Phishing Problem

Phishing is a cyberattack in which the attacker pretends to be a trusted person, company, or service to trick a victim into giving up sensitive information such as passwords, one-time passwords (OTPs), card details, or money. It has expanded well beyond email into text messages (smishing), voice calls (vishing), fake websites, and social media. As digital banking and mobile financial services (MFS) grow rapidly in countries like Bangladesh, the impact of a successful phishing attack is severe. A single compromised MFS account can wipe out the savings of a low-income user.

The problem shows up in everyday situations. A user receives an SMS that looks like it came from their bank and asks them to verify their account through a suspicious link. Another user gets a phone call from someone pretending to be customer support and asking for an OTP. A third user lands on a fake login page that looks visually identical to a real banking website. In each case the content feels normal, but the underlying URL, sender, or audio source is malicious, and most users do not have the technical background to spot the difference on their own.

2.2 Why Existing Solutions Are Not Enough

Traditional protections fall into three groups. First, blacklist-based tools (such as built-in browser warnings) only block sites that are already known to be malicious. They cannot stop newly created phishing pages, which often live for only a few hours before being taken down. Second, classical machine learning detectors trained on English data fail when the phishing content is in Bangla, mixed Bangla-English, or hidden inside images. Third, most existing systems focus on only one type of input. A URL-based detector cannot read an image, an image-based detector cannot understand the language used, and a text-based detector cannot inspect a URL.

These limitations have been documented in recent academic work. Reviews of phishing detection methods between 2017 and 2024 show that hybrid and multi-modal models consistently outperform single-modality models, and that research is moving toward explainable AI, lightweight LLMs, and unsupervised anomaly detection [11, 10, 14]. Studies on mobile financial services in Bangladesh and other emerging markets stress the need for systems that handle local languages, voice phishing, and OTP-bypass social engineering [3, 8]. Telecommunication studies show that even SMS phishing detection can benefit from advanced feature extraction and CNNs [1].

2.3 Why an Anomaly-Based Approach Is Needed

A second major issue is that phishing attacks keep evolving. Attackers constantly create new domains, new templates, and new scripts, so a model trained on yesterday's phishing examples may not recognize today's attack. This is the zero-day problem. To handle it, the proposed system uses an anomaly detection engine based on the Isolation Forest algorithm, an unsupervised method that learns what "normal" looks like and flags anything that deviates from it. Recent work shows that Isolation Forest and its deep variant work well on high-dimensional and non-linear data while remaining light enough for real-time use [18, 19], which makes the algorithm a good fit for a browser-based phishing detector where speed is critical and labeled data for new attacks is rare.

2.4 Project Motivation

Combining multi-modal detection with anomaly-based intelligence gives a defense that is both wide and adaptive at the same time. It is wide because the system can look at URLs and screenshot text together rather than relying on just one signal. It is adaptive because the Isolation Forest engine can flag new and unusual patterns without needing to be retrained for every fresh attack. That combination, which is missing in most existing tools, is what motivates PhishGuard. The project also targets Bangla and Bangla-English mixed content, which is a real-world need for users in Bangladesh that most international tools do not serve.

Overall, this capstone brings together engineering knowledge across software development, machine learning, anomaly detection, natural language processing, and cybersecurity to address a real-life complex problem that affects millions of users every day.

3. Related Works

3.1 State-of-the-Art Comparison Table

No.	Paper / Authors	Main Concept	Coverage Area	Limitation
1	Hapase & Patil Telecom Fraud Resilient Framework for SMS Phishing using AI	Enhanced CNN with PCC-PCA feature extraction for SMS phishing in telecom systems.	SMS phishing, telecom fraud, AI classification, CNN.	Mainly SMS dataset; limited multilingual and real deployment testing.
2	Mollik & Majeed AI-Driven Cybersecurity in Mobile Financial Services	AI-driven fraud detection and privacy enhancement in MFS in emerging markets.	Mobile financial services, privacy, explainable AI, user trust.	Survey-based; lacks technical implementation.
3	Sarker Integrated Fraud Defense for Card-Based Account Funding in Bangladesh MFS	Socio-technical framework using Fraud Triangle, RAT, and NIST CSF.	OTP bypass, MFS fraud, Bangladesh fintech.	Conceptual only; no implementation or experimental validation.

No.	Paper / Authors	Main Concept	Coverage Area	Limitation
4	Ambore et al. Resilient Cybersecurity Framework for MFS	Resilient cybersecurity framework for securing MFS ecosystems.	MFS, human factors, cybercrime mitigation.	Preliminary; limited experimental validation.
5	Ononiwu et al. ML for Fraud Detection in Mobile Banking and Fintech	ML models for fraud detection and risk assessment in fintech.	Mobile banking, fintech, anomaly detection, risk.	Review-oriented; lacks deployment and benchmarking.
6	Ngo et al. Secure Mobile Banking with AI-Driven Fraud Detection	Mobile banking system with biometrics and AI fraud detection.	AI fraud detection, biometrics, anomaly detection.	Prototype-only; weak on scalability and adversarial attacks.
7	Sayibu et al. Fraud in Mobile Money Payment Systems: SLR of Text-Based Methods	Systematic review of text-based fraud detection in mobile money.	Mobile money fraud, SMS fraud, NLP detection.	No new model proposed; only literature synthesis.
8	MIST/MBSTU Vishing Shield Framework for MFS	Conceptual vishing detection model for voice phishing in MFS.	Vishing, social engineering, user awareness.	Conceptual and survey-based; no real implementation.
9	Williams Hybrid ML for Adaptive Fraud Prevention in Mobile Banking	Hybrid ML combining supervised, unsupervised, and reinforcement learning.	Adaptive fraud prevention, ensembles, RL.	Theoretical; limited experimentation and dataset diversity.
10	Buiya Systematic Review of AI-Enabled Fraud Detection	Comparative review of ML, DL, hybrid, ensemble, graph-based methods.	Digital finance fraud, DL, graphs, ensembles.	Secondary research; no original framework.
11	Wilk-Jakubowski et al. ML & NN for Phishing Detection (2017–2024)	Review of ML and NN approaches for phishing across channels.	Website, email, social media phishing; ML, DL.	Review-based; no new detection model proposed.

No.	Paper / Authors	Main Concept	Coverage Area	Limitation
12	Zara et al. Phishing Website Detection Using Deep Learning Models	ML, ensembles, PCA, and DL for phishing website detection.	Website phishing, ensembles, feature selection.	Dataset-driven; real-time deployment not discussed.
13	Barik et al. Web-Based Phishing URL Detection with DL Optimization	EGSO-CNN optimized DL model for phishing URL detection.	URL phishing, optimization, feature engineering.	High compute cost; depends heavily on dataset quality.
14	Popescul & Radu AI in Phishing Detection: Bibliometric Review	Bibliometric review of AI/ML for phishing research.	AI phishing, bibliometric, NLP, DL.	Does not provide practical implementation.
15	Sim & Kim Voice Phishing Detection with Fine-Tuned SLMs	Voice phishing detector using fine-tuned Llama3-based small LLMs.	Voice phishing, small LLMs, fine-tuning.	Dependent on transcript quality and language-specific data.
16	Thapa et al. Phishing in the Gen-AI Era: Quantized LLMs vs Classical Models	Comparison of ML, DL, and quantized LLMs for phishing.	GenAI, quantized LLMs, robustness, explainability.	LLMs still underperform classical ML/DL on accuracy.
17	Rao et al. Hybrid Super Learner Ensemble (Phish-Jam) for Mobile Devices	Hybrid ensemble mobile phishing detection app.	Mobile phishing, ensembles, transformers.	Mobile resource limits; mostly URL features.
18	Xu et al. Deep Isolation Forest for Anomaly Detection	DIF combines isolation forests with deep neural random representations.	Anomaly detection, isolation forest, DL, time-series.	Compute-heavy; complex for real-time deployment.
19	Fadul Isolation Forest vs Local Outlier Factor for Anomaly Detection	Comparison of IForest vs LOF across multiple datasets.	Anomaly detection, IForest, LOF, fraud detection.	Only two algorithms; no hybrid or deep methods.

No.	Paper / Authors	Main Concept	Coverage Area	Limitation
20	Liu et al. Isolation Forest	Original isolation forest algorithm for unsupervised anomaly detection.	Anomaly detection foundational algorithm.	Designed pre-deep-learning; limited handling of non-linear feature spaces.

Table 3.1; Comparative review of twenty papers on fraud detection, phishing detection, and mobile financial security.

3.2 Summary of Observations

Several clear patterns emerge from the twenty papers reviewed. First, AI, machine learning, deep learning, and hybrid models dominate the research landscape, and hybrid or ensemble systems consistently outperform single-model systems in fraud and phishing detection [10, 11, 17]. Second, mobile financial services security is a strong research focus, particularly in emerging markets such as Bangladesh, where social engineering attacks like OTP bypass and vishing are common [2, 3, 4, 8]. Third, social engineering attacks (phishing, smishing, vishing) are growing faster than purely technical attacks, and they require detection systems that understand language and context, not just URLs [1, 7, 15].

Fourth, explainable AI, lightweight LLMs, and adaptive learning are emerging future directions [16, 9]. Fifth, many papers stop at conceptual frameworks or systematic reviews and do not provide working systems [3, 4, 7, 10, 11, 14]. Sixth, real-time scalability, adversarial robustness, multilingual support, and privacy preservation remain major limitations across studies. Seventh, anomaly detection methods such as Isolation Forest and its deep variant are mature and well-suited for detecting unknown attacks, but few phishing-focused systems integrate them with multi-modal pipelines [18, 19, 20].

3.3 Justification for the Proposed Method

The literature gap points to a clear opportunity. No reviewed paper provides all of the following at once:

1. a working prototype.
2. multi-modal coverage including URLs and screenshot text extracted through OCR.
3. Bangla and English support.
4. real-time browser-based deployment.
5. anomaly-based detection of unknown attacks.

PhishGuard is proposed to fill this gap. It integrates supervised models (NLP classifier, URL classifier) for known phishing patterns with an unsupervised Isolation Forest engine for zero-day patterns. It is delivered as a browser extension, which addresses the real-time scalability issue. It is built with Bangla support from the start, which addresses the multilingual gap. And it follows privacy-by-design principles, which addresses the privacy concern raised in [2] and reinforced in modern data protection law.

4. Research Questions and Problem Statements

4.1 Problem Statement

Despite many years of research into phishing detection, users in countries like Bangladesh remain highly vulnerable. The core problem is that current phishing defenses are narrow, slow to adapt, and language-limited. They either focus on one input type (only URLs or only emails), only work on English content, only flag known threats, or only exist as research prototypes without a real-time user-facing tool. As a result, even a moderately skilled attacker can defeat existing protections by using a fresh domain, a Bangla phishing message, or an image-based scam page. The capstone problem is therefore to design and prototype a single system that detects phishing across multiple modalities (URL and screenshot text extracted through OCR), supports Bangla and English, runs in real time, and can flag unknown patterns through anomaly detection, all while respecting user privacy.

4.2 Main Research Question

RQ-Main: How can a multi-modal phishing detection system, enhanced by Isolation Forest anomaly detection, accurately and quickly identify phishing attempts across URLs and OCR-extracted screenshot text in both Bangla and English while running as a real-time browser extension?

4.3 Sub-Questions

RQ-1: Which combination of OCR and NLP techniques gives reliable detection of phishing content from screenshots containing Bangla, English, and mixed-language text?

RQ-2: Which URL features (lexical, host-based, reputation-based) are most predictive of phishing in newly observed domains, and how should they be combined with anomaly detection for zero-day URLs?

RQ-3: How well does Isolation Forest work as an unsupervised anomaly detector for URL features and OCR text features compared to supervised baselines?

RQ-4: What system architecture allows all of the above components to run in real time inside a browser extension while preserving user privacy and consent?

RQ-5: How can multiple phishing detection signals be fused into a unified real-time phishing risk score for browser-based alerting?

5. Objectives

5.1 Overall Aim

The overall aim of this study is to design, implement, and evaluate PhishGuard, a multi-modal phishing detection system that combines supervised machine learning with unsupervised Isolation Forest anomaly detection, supports Bangla and English, and runs as a real-time browser extension with a backend analysis engine. The core scope is bound to URL analysis and OCR + NLP, supported by an Isolation Forest anomaly engine. Voice phishing (transcript-only experimental study), advanced behavior monitoring, and a network-layer analysis module are treated as bounded future extensions (Section 6.8).

5.2 Specific Objectives (Core)

- 1) Objective 1 OCR + NLP: Design and build an OCR plus NLP pipeline that can extract text from Bangla and English screenshots and classify the text as phishing or benign with an F1-score in the range 0.85–0.90.
- 2) Objective 2 URL Analysis: Build a URL analysis module that extracts lexical and host-based features and combines a supervised classifier with an Isolation Forest anomaly detector to flag suspicious URLs with an F1-score in the range 0.88–0.92.
- 3) Objective 3 Anomaly Engine: Integrate an Isolation Forest based anomaly detection engine that runs across URL features and OCR text features to catch zero-day attacks.
- 4) Objective 4 Browser Extension + Backend: Develop a browser extension front-end with a Python FastAPI backend, integrate all core modules, and implement a real-time alert system targeting end-to-end latency of ≤ 5 seconds.
- 5) Objective 5 Evaluation: Evaluate the full system using precision, recall, F1-score, false positive rate, and latency on a held-out test set, and conduct a user acceptance test to verify usability.
- 6) Objective 6 Privacy: Ensure that the system follows privacy-by-design principles consistent with GDPR Article 6(1)(a), including explicit user consent, data minimization, and on-device processing where possible.

6. Planned Methodology

This section presents the planned methodology of PhishGuard. It first discusses the strengths and weaknesses of candidate methods and justifies the chosen approach. It then describes the proposed system architecture, data plan, algorithms, evaluation metrics, the role of the Isolation Forest anomaly engine inside the pipeline, and an explicit Implementation Boundary that separates the core deliverable from experimental and future-work extensions. The methodology is designed to directly answer the research questions in Section 4 and meet the objectives in Section 5.

6.1 Choice of Method and Justification

Several alternative approaches were considered for each module. URL-only detection is fast but misses image-based scams. OCR alone is useful for text in images but does not understand context or domain reputation. Supervised classifiers handle known patterns well but fail on zero-day attacks. Unsupervised anomaly detection covers new attacks but is noisy on its own. Therefore the proposed method is a hybrid pipeline: a supervised text classifier (mBERT) and a supervised URL classifier (Random Forest) for known patterns, plus Isolation Forest for anomalies, fused at a decision layer. Screenshot-based computer vision was considered and was deliberately excluded from the core capstone to keep scope realistic; image-based scams are partially covered by the OCR + NLP path.

Table 6.1 compares the main choices.

Table 6.1 Method Comparison and Final Selection

Module	Option A (Selected)	Option B	Option C	Why Selected
OCR Engine	Tesseract (open source, Bangla support)	Google Cloud Vision	EasyOCR	Free, on-device, strong Bangla support.
Text Classifier	Fine-tuned mBERT (Bangla + English)	BanglaBERT / XLM-R	Classical ML (SVM/RF)	Covers Bangla and English in one model; high accuracy on mixed text. The NLP classifier will be implemented using Hugging Face Transformers with a fine-tuned multilingual BERT (mBERT) model.
URL Classifier	Random Forest + Isolation Forest	XGBoost only	Rule-only	Combines supervised accuracy with zero-day anomaly detection.
Anomaly Detector	Isolation Forest (sklearn)	Local Outlier Factor	Autoencoder	Lightweight, fast, unsupervised, real-time friendly [18, 19, 20].
Backend	Python FastAPI	Node.js Express	Flask	Async, fast, integrates well with Python ML libraries.

Module	Option A (Selected)	Option B	Option C	Why Selected
Frontend	React + Manifest V3 extension	Vue.js	Plain JS	Mature ecosystem, easy state management, modern security model.

Table 6.1 : Comparison of candidate technologies and the reason each one was selected for PhishGuard .

6.2 Proposed System Architecture

PhishGuard has three logical layers:

- (i) a client layer in the form of a browser extension that captures URLs and screenshots;
- (ii) a backend layer running a FastAPI server that exposes endpoints for each detection task
- (iii) a machine learning and anomaly detection layer that hosts the URL, OCR, NLP, and Isolation Forest modules.

The user-facing layer is intentionally thin so that the extension stays lightweight. Heavy inference happens on the backend. Figure 6.1 shows the high-level multi-modal pipeline (core only). Figure 6.2 shows the Isolation Forest anomaly engine. Figure 6.3 shows the request flow between extension and backend.

Figure 6.1 High-Level Multi-Modal Pipeline (Core)

URL	Screenshot (text region)
↓	↓
URL Feature Extractor	OCR + NLP
↓	↓
URL Feature Vector	Text Feature Vector
↓	↓
Isolation Forest Anomaly Engine + Decision Fusion Layer	
↓	
Risk Score → Real-Time Alert in Browser Extension (Green / Yellow / Red)	

Figure 6.1 : Core pipeline. URL and screenshot text are converted into feature vectors. The supervised URL classifier and OCR + NLP classifier run alongside the Isolation Forest anomaly engine, and the Decision Fusion Layer produces a final risk score that the browser extension renders as a Red, Yellow, or Green alert.

Figure 6.2 Isolation Forest Anomaly Engine (Core)

URL Features	OCR Text Features
↓	↓
Isolation Forest (Unsupervised) builds many random trees; anomalies isolated in fewer splits	
↓	
Normal (low anomaly score)	Anomaly (high anomaly score)
↓	
Send anomaly score to Decision Fusion Layer	

Figure 6.2 : The core Isolation Forest engine consumes only URL features and OCR text features. It builds random trees and assigns an anomaly score to each input. High-score samples are flagged even if they were never seen during training. Adding screenshot layout features.

Figure 6.3 Request Flow Between Components

User Browser	PhishGuard Extension	FastAPI Backend	ML / Anomaly Modules
→ URL / Screenshot	→ HTTPS request	→ Dispatch by type	→ Inference
← Red / Yellow / Green Alert	← JSON risk score	← Fused result	← Per-module scores

Figure 6.3 : End-to-end flow. The browser extension sends URLs or screenshots over HTTPS. The FastAPI backend dispatches the request to the right ML or anomaly module and returns a fused JSON risk score, which the extension renders as a Red, Yellow, or Green alert.

6.3 Module-Level Methodology

6.3.1 OCR + NLP Module (Core)

The screenshot is captured by the browser extension using the standard captureVisibleTab API. The image is preprocessed (deskew, denoise) and passed through Tesseract OCR with both English and Bangla language packs to extract text. The extracted text is cleaned (Unicode normalization, basic tokenization, lower-casing for English) and passed into a fine-tuned multilingual BERT classifier. The classifier is implemented using the Hugging Face Transformers library. The classifier outputs a phishing probability between 0 and 1. This addresses RQ-1.

6.3.2 URL Analysis Module (Core)

Every URL the user visits or pastes is parsed and the following features are extracted: length, number of dots, number of hyphens, presence of suspicious tokens (login, verify, secure, otp, bank), entropy of the path, use of IP instead of domain, TLD category, HTTPS presence, domain age (via WHOIS where available), and a lookup against VirusTotal and Google Safe Browsing. These features feed a Random Forest classifier for known patterns and an Isolation Forest model for anomaly scoring. This addresses RQ-2.

6.3.3 Isolation Forest Anomaly Engine (Core)

This is the adaptive backbone of the system. The Isolation Forest algorithm builds many random binary trees on a sample of normal feature vectors. Anomalies require fewer splits to be isolated, so they end up with shorter average tree paths and higher anomaly scores. In the core implementation, PhishGuard runs Isolation Forest on two feature streams: URL features and OCR text features (TF-IDF and length-based features). Isolation Forest is chosen because it is lightweight, fast, unsupervised, and well-suited to real-time browser-extension use [18, 19, 20]. An optional Deep Isolation Forest variant [18] is documented as a possible future upgrade but is not part of the core deliverable. This addresses RQ-4.

6.3.4 Decision Fusion and Real-Time Alerting (Core)

Each core module produces a score between 0 and 1. The Decision Fusion Layer combines them using weighted voting: the final risk score is a weighted average where supervised models receive a higher weight than the anomaly score by default, with weights tuned on a validation set. The Decision Fusion Layer combines URL classification scores, OCR + NLP scores, and Isolation Forest anomaly scores into a unified phishing risk score. If the supervised path or the anomaly path strongly flags the input, the system raises the alert level. The extension shows three levels to the user: Green (safe), Yellow (suspicious), and Red (likely phishing). This addresses RQ-5.

6.3.5 Voice / Transcript Module (Bounded Experimental Extension)

Voice phishing support is proposed as a future extension. The current CSE400A/CSE400B implementation will focus on transcript-based experimental analysis only there is no live audio capture, no audio model training, and no deepfake voice detection. The goal of the experiment is narrow: given a Bangla or English call transcript (produced off-line by an existing speech-to-text tool such as OpenAI Whisper), can the NLP classifier trained for smishing text also detect vishing-style scam language? Typical examples of the scam language being detected are: “Sir, OTP bolun account verify korte hobe,” “Dear customer, please share your card PIN to reactivate your account,” or “Apnar bKash account block hoye geche, taratari amake call koren.”

This experiment reuses the same mBERT classifier from Section 6.3.1; no new model is trained. Live in-call voice analysis, on-device speech recognition, deepfake / synthetic-voice detection, and any MFCC + CNN-BiLSTM audio model are explicitly out of scope. This bounded design lets the project investigate RQ-6 without inflating the capstone scope.

6.4 Evaluation Metrics and Targets

To evaluate each module and the overall system, the following metrics are used: precision, recall, F1-score, ROC-AUC, false positive rate, and end-to-end latency. Targets in Table 6.2 are intentionally set at realistic ranges that are consistent with reported numbers in the literature for similar tasks, so that they can be defended during evaluation.

Table 6.2 Evaluation Targets

Module	Primary Metric	Target Range	Acceptable Latency
OCR + NLP (Core)	F1-score (phishing class)	F1: 0.85 – 0.90	≤ 3.0 s per screenshot
URL Classifier (Core)	F1-score and AUC	F1: 0.88 – 0.92; AUC ≥ 0.92	≤ 1.0 s per URL
Isolation Forest (Core)	AUC	AUC ≥ 0.80	≤ 0.3 s per record
System (End-to-End, Core)	False Positive Rate; latency	FPR $\leq 7\%$	≤ 5.0 s end-to-end

Table 6.2 : Revised quantitative evaluation targets per module and for the full system. Ranges replace single-point targets to give realistic, defensible numbers.

These ranges are aligned with the performance reported in the literature for similar tasks: multilingual BERT classifiers reach around 0.85–0.90 F1 on Bangla and mixed-language phishing/smishing text [1, 8]; classical ML / ensemble URL classifiers reach 0.88–0.92 F1 on PhishTank-style data [12, 13, 17]; and Isolation Forest reaches strong AUC values on tabular anomaly tasks [19, 20]. End-to-end latency of ≤ 5 seconds is consistent with running a Transformer classifier behind an HTTPS call on a small server.

6.4.1 How the Targets Will Be Reached

The numerical ranges in Table 6.2 are not picked from thin air. Each module has a concrete plan that makes the target plausible. The supervisor or examiner can reasonably ask, “How will you actually reach F1 of 0.88–0.92 on the URL classifier?” the answer below is what the project plans to do.

For the URL classifier (target F1: 0.88–0.92), four things together make this range realistic. First, the training set will be balanced: the same number of phishing and benign URLs are used, with stratified sampling so that the validation and test sets keep the same balance. Second, the feature set is engineered rather than raw a mix of lexical features (length, dots, hyphens, suspicious tokens, path entropy), host-based features (IP-vs-domain, TLD category, HTTPS, domain age via WHOIS), and reputation features (VirusTotal, Google Safe Browsing lookups). Random Forest handles this kind of mixed tabular feature set well and resists overfitting through bagging. Third, hyperparameters

(number of trees, max depth, min samples per leaf) are tuned by 5-fold cross-validation on the training split. Fourth, the Isolation Forest anomaly score is concatenated as an additional feature, so the classifier benefits from both supervised and unsupervised signals. Published F1 scores for similar URL classifiers on PhishTank-style data sit in roughly the same range [12, 13, 17], so the target is consistent with the literature.

For the OCR + NLP classifier (target F1: 0.85–0.90), the plan combines a clean OCR pipeline (Tesseract with both Bangla and English language packs, deskew, denoise) with a fine-tuned mBERT classifier that already understands Bangla, English, and code-switched text out of the box. Fine-tuning is done on the smishing/phishing text corpora described in Table 7.1, with stratified splits and a small amount of synthetic augmentation for class balance. Reported F1 scores for mBERT-class models on Bangla smishing detection sit around 0.85–0.90 [1, 8], which is the target range.

For the Isolation Forest engine (target AUC ≥ 0.80), the model is fit only on benign feature vectors so the engine learns what “normal” looks like. The contamination ratio, number of estimators, and sample size are tuned on a validation set that contains a controlled mix of normal and held-out anomalous samples. AUC of 0.80–0.85 is a conservative target compared to results reported in [19, 20].

For the end-to-end latency target (≤ 5 seconds), the design choices that protect it are: a thin browser extension that defers heavy work to the backend; asynchronous FastAPI request handling; running OCR and the mBERT classifier sequentially per request rather than running every module on every input; and parallel calls to reputation APIs. Removing the screenshot vision module from core scope also makes this target more comfortable.

Finally, an honest fallback applies to every target: if a module misses its range during CSE400B evaluation, the report will state the achieved number, explain the gap, and the project will still be judged on the strength of its architecture, scope discipline, and end-to-end working prototype. The targets in Table 6.2 guide the work; they do not lock the project into a specific number.

6.5 Security, Privacy, and Compliance

PhishGuard is designed to follow privacy-by-design. The extension requests explicit user consent on installation. Screen capture is off by default and only runs after the user enables it. All transmission between extension and backend uses HTTPS. Logs are anonymized and retention is short. The design follows GDPR Article 6(1)(a) for consent and the data minimization principle, consistent with general European data protection guidelines and Bangladesh Digital Security Act expectations. This addresses Objective 7.

6.6 Tools and Implementation Plan

The implementation stack is intentionally kept small to suit a capstone scope. The backend uses Python 3.x with FastAPI, PyTorch, scikit-learn, and Hugging Face Transformers. The browser extension is built with JavaScript and React (Chrome Manifest V3, also targeting Firefox WebExtensions). Tesseract is used for OCR. Trained model weights, a small URL whitelist, and a reputation cache are stored as local files; a database server is not required for the core deliverable. The backend runs either locally on the developer machine during testing.

6.7 Implementation Boundary (Core System vs Future Work)

To prevent scope inflation during the capstone, the project is explicitly partitioned into a core implementation that will be built, tested, and defended, and a set of experimental or future-work extensions that are documented but not built end-to-end. Table 6.3 summarises this partition.

Table 6.3 Implementation Boundary

Component	Status	Justification
URL phishing detection	Core	Mature literature, well-defined feature set, fits real-time browser use.
OCR + NLP (Bangla + English)	Core	Addresses the multilingual gap in existing tools; tractable with mBERT + Tesseract.
Isolation Forest anomaly engine (URL + OCR text)	Core	Adds zero-day coverage; lightweight enough for real-time backend.
Browser extension + FastAPI backend	Core	Required to make the system deployable, not just a research artifact.

7. Data Analysis Plans

This section describes how data will be collected, prepared, and analyzed across the core modalities. Since the project mixes supervised and unsupervised learning, the data plan covers both labeled and unlabeled streams. Voice transcripts are described separately because they belong to the bounded experimental extension.

7.1 Data Sources

Modality	Primary Sources	Notes
Phishing URLs (Core)	UCI Phishing Websites dataset (https://archive.ics.uci.edu/dataset/327/phishing+websites) , VirusTotal lookups(https://www.virustotal.com/gui/home/url) , Common crawl(https://commoncrawl.org/) , Tranco(https://tranco-list.eu/)	Used for supervised URL classifier and Isolation Forest baseline.
Bangla SMS / Smishing (Core)	BanglaBarta SMS dataset (https://github.com/search?q=BanglaBarta+dataset&type=repositories) ; Bangla phishing/SMS datasets on Kaggle (https://www.kaggle.com/search?q=bangla+sms+dataset)	Trains mBERT for Bangla phishing text.
English Phishing Text (Core)	Kaggle phishing email datasets (https://www.kaggle.com/datasets/subhajournal/phishingemails), (https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset) phishing-related text from UCI (https://archive.ics.uci.edu/dataset/228/sms+spam+collection)	Augments mBERT training for English coverage.
Voice Transcripts (Experimental extension)	Public vishing call corpora where available; volunteer-recorded Bangla and English vishing scripts; transcribed off-line. Possible English scam-call sources for transcript experiments (https://www.youtube.com/results?search_query=scam+call+recording) , Possible voice datasets (https://www.kaggle.com/search?q=scam+call+audio)	Used only for the bounded transcript experiment in Section 6.3.6.

Table 7.1 : Data sources, labeling strategy, and intended use for each modality.

7.2 Data Preprocessing

Text data (OCR output and, for the experiment, call transcripts) is cleaned by Unicode normalization, removal of noise characters, basic tokenization, and lower-casing for English. Bangla text uses a Bangla-aware tokenizer. URLs are parsed into components (scheme, host, path, query) and numeric features are extracted and scaled. Screenshots captured by the extension are processed only by the OCR pipeline (deskew, denoise, language pack selection) ; there is no separate image classifier in the core scope, so no resizing or ImageNet normalization is needed. For the bounded voice experiment, audio is resampled to 16 kHz and converted to a Whisper-compatible format off-line; no MFCC feature extraction or audio-model training is performed.

7.3 Labeling and Annotation

Where labels are not available from the source, two human annotators label items independently. Disagreements are resolved by a third reviewer. Special attention is paid to Bangla-English code-switched messages, which are common in Bangladesh and important for real-world phishing scenarios.

7.4 Splits and Cross-Validation

Each labeled dataset is split into 70% training, 15% validation, and 15% test. Stratified sampling preserves class balance. For URLs and tabular features, 5-fold cross-validation is used during hyperparameter tuning. For Isolation Forest, the model is fit on the “normal” portion of the training data only, and the test set includes a controlled mix of normal and anomalous samples so that AUC and precision at top-k can be measured.

7.5 Analysis Methods

For supervised classifiers, the analysis computes precision, recall, F1-score, accuracy, and ROC-AUC. Confusion matrices are reported per class. For Isolation Forest, anomaly score distributions are visualized; the operating threshold is tuned on the validation set to balance recall and false positive rate. For the fused decision layer, weights are tuned by grid search on the validation set against the F1-score of the phishing class. Latency is measured end-to-end and per module on a representative test machine.

7.6 Ethical and Privacy Considerations

All user-generated data (voice samples for the bounded experiment, screenshots used during user testing) is collected with informed consent. No personally identifying information is stored beyond what is needed for the test. Screenshots used during evaluation are scrubbed of sensitive content before being added to the dataset. The data plan follows the GDPR principle of data minimization throughout the project lifecycle.

8. Expected Results

8.1 Functional Outcomes

By the end of CSE400B, PhishGuard is expected to function as a working real-time phishing detection tool for its core scope. A user installs the browser extension, opts in to the features they want (basic URL check is always on; screenshot scan is opt-in), and receives a visible alert when they encounter a phishing attempt. The system handles Bangla and English content in screenshots through the OCR + NLP path and supports common phishing URL patterns through the URL classifier and the Isolation Forest anomaly engine. The voice-transcript experiment is reported as a separate study and does not appear as a live alert path

8.2 Quantitative Targets

Each core module is expected to meet the ranges in Table 6.2. In particular, the URL classifier is expected to reach F1 in the range 0.88–0.92, and the OCR + NLP classifier is expected to reach F1 in the range 0.85–0.90. The Isolation Forest engine is expected to reach an AUC of at least 0.80 on held-out anomalies, demonstrating that it adds real value on top of the supervised models. The end-to-end false positive rate is targeted at 7% or lower so that the extension does not annoy users, and end-to-end latency on a standard machine is targeted at 5 seconds or less.

8.3 Qualitative Outcomes

Beyond raw numbers, the project is expected to demonstrate that multi-modal and anomaly-based detection together produce a more resilient defense than any single approach in the literature, even within a deliberately bounded scope. The user acceptance test is expected to confirm that real users find the alerts understandable and the system unobtrusive. The project should also produce reusable artifacts (datasets, trained model

weights, code) that other students or researchers in Bangladesh can extend, including the documented future-work extension surface.

8.4 Planned Deliverables

Deliverable	Description	Phase
Browser extension prototype (Core)	Chrome / Firefox extension with real-time alerts (Green / Yellow / Red) for URL and screenshot.	CSE400B
Backend API (Core)	FastAPI server hosting OCR, NLP, URL, and Isolation Forest modules	CSE400B
Trained ML models (Core)	Fine-tuned mBERT, URL Random Forest, Isolation Forest model.	CSE400B
Dataset package (Core)	Cleaned and labeled datasets for URLs, Bangla/English text, and screenshots.	CSE400A → CSE400B
Evaluation report (Core)	Per-module and end-to-end metrics: F1, AUC, FPR, latency.	CSE400B
Voice-transcript experimental note (Extension)	Short experimental report on transcript-only vishing detection using the trained NLP classifier.	CSE400B (if time permits)
Future-work architecture document	Schema and integration plan for behavior monitoring and Deep Isolation Forest upgrades.	CSE400C
Final capstone report	Full written report with architecture, methodology, evaluation, and references.	CSE400C

Table 8.1 : Planned deliverables and the phase in which each is produced.

8.5 Contribution

If the targets above are met, the contribution of this work is threefold. First, it produces one of the first multi-modal phishing detection systems with full Bangla and English support targeted at users in Bangladesh, even within a deliberately bounded core scope. Second, it shows in a practical setting how Isolation Forest can be combined with supervised models on URL and OCR text features to handle zero-day phishing patterns. Third, it produces a working browser-deployable prototype, which is rare in the reviewed

literature where most contributions stop at concepts or offline experiments. A clearly documented future-work surface (Section 6.8, Figure 6.4) makes it straightforward for follow-on work to extend the system into voice and behavior modalities without re-architecting it.

REFERENCES:

- [1] D. S. Hapase and L. V. Patil, “Telecommunication fraud resilient framework for efficient and accurate detection of SMS phishing using artificial intelligence techniques,” *Multimedia Tools and Applications*, vol. 83, pp. 89111–89133, 2024, doi: 10.1007/s11042-024-19020-2.
- [2] E. Mollik and F. Majeed, “AI-driven cybersecurity in mobile financial services: Enhancing fraud detection and privacy in emerging markets,” *Journal of Cybersecurity and Privacy*, vol. 5, no. 3, art. 77, 2025, doi: 10.3390/jcp5030077.
- [3] S. K. Sarker, “An integrated fraud defense framework for card-based account funding in Bangladesh’s mobile financial services: Addressing social engineering-driven OTP bypass through Fraud Triangle, Routine Activity, and NIST Cybersecurity principles,” *ResearchGate preprint*, Nov. 2025. [Online]. Available: <https://www.researchgate.net/publication/397198404>
- [4] S. Ambore, C. Richardson, H. Dogan, E. Apeh, and D. Osselton, “A resilient cybersecurity framework for mobile financial services (MFS),” *Journal of Cyber Security Technology*, vol. 1, no. 3–4, pp. 202–224, 2017, doi: 10.1080/23742917.2017.1386483.
- [5] M. Ononiwu, T. I. Azonuche, O. F. Okoh, and J. O. Enyejo, “Machine learning approaches for fraud detection and risk assessment in mobile banking applications and fintech solutions,” *International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET)*, vol. 10, no. 4, 2023, doi: 10.32628/IJSRSET232531.
- [6] A. Ngo, S. Sapaev, F. Jumaniyozov, Z. Mamadiyarov, A. Mittal, and I. Jurayev, “Design and deployment of a secure mobile banking system with AI-driven fraud detection,” in *Proc. IEEE International Conference*, 2025, doc. 11210372. [Online]. Available: <https://ieeexplore.ieee.org/document/11210372>
- [7] U. Sayibu, M. Asante, G. Abdul-Salam, and T. Frimpong, “Fraud prediction and prevention in mobile money payment systems (MMPS): A systematic literature review of

text-based detection methods,” *Security and Communication Networks*, vol. 2025, art. 8913715, 22 pp., 2025, doi: 10.1155/sec/8913715.

[8] M. W. Chy, M. A. Kabir, M. E. Tarek, M. A. Hossain, M. S. Uddin et al., “Vishing Shield: A conceptual framework and detection model to mitigate vishing attacks in mobile financial services (MFS),” *ResearchGate working paper*, MIST/MBSTU, 2024.

[9] Replace this reference because the original citation could not be verified. Suggested replacement:

I. Bello, A. Ige, and D. Ameyaw, “Adaptive machine learning models: Concepts for real-time financial fraud prevention in dynamic environments,” *World Journal of Advanced Engineering Technology and Sciences*, vol. 12, no. 2, pp. 21–34, 2024.

[10] M. R. Buiya, “A systematic review of AI-enabled fraud detection in digital financial systems (2019–2026),” *American Journal of Data Science and Analytics*, vol. 7, no. 3, pp. 125–162, 2026, doi: 10.63125/wpj89816.

[11] J. L. Wilk-Jakubowski, L. Pawlik, G. Wilk-Jakubowski, and A. Sikora, “Machine learning and neural networks for phishing detection: A systematic review (2017–2024),” *Electronics*, vol. 14, no. 18, art. 3744, 2025, doi: 10.3390/electronics14183744.

[12] U. Zara, K. Ayub, H. U. Khan, A. Daud, T. Alsahfi, and S. Gulzar, “Phishing website detection using deep learning models,” *IEEE Access*, vol. 12, pp. 167072–167087, 2024.

[13] K. Barik, S. Misra, and R. Mohan, “Web-based phishing URL detection model using deep learning optimization techniques,” *International Journal of Data Science and Analytics*, vol. 20, pp. 4449–4471, 2025, doi: 10.1007/s41060-025-00728-9.

[14] D. Popescul and L. D. Radu, “AI in phishing detection: A bibliometric review,” *Frontiers in Artificial Intelligence*, vol. 8, art. 1496580, 2025, doi: 10.3389/frai.2025.1496580.

[15] J. Y. Sim and S. H. Kim, “Detecting voice phishing with precision: Fine-tuning small language models,” *arXiv preprint arXiv:2506.06180*, 2025.

[16] J. Thapa, G. Chahal, Ș. V. Gabreanu, and Y. Otoum, “Phishing detection in the Gen-AI era: Quantized LLMs vs classical models,” *arXiv preprint arXiv:2507.07406*, 2025.

- [17] R. S. Rao, C. Kondaiah, A. R. Pais, and B. Lee, “A hybrid super learner ensemble for phishing detection on mobile devices,” *Scientific Reports*, vol. 15, 2025, doi: 10.1038/s41598-025-02009-8.
- [18] H. Xu, G. Pang, Y. Wang, and Y. Wang, “Deep Isolation Forest for anomaly detection,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12591–12604, 2023, doi: 10.1109/TKDE.2023.3270293.
- [19] A. M. A. Fadul, “Anomaly detection based on Isolation Forest and Local Outlier Factor,” Master’s essay, African Institute for Mathematical Sciences (AIMS) South Africa, May 2023.
- [20] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. 8th IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

Appendix: Addressing of COs, Knowledge Profile (K), and Complex Engineering Problem (EP)

This appendix maps each Course Outcome (CO) of CSE400A onto the relevant report sections and indicates which Knowledge Profile attributes (K1–K8) and Complex Engineering Problem attributes (EP1–EP7) are addressed. The mapping covers all CSE400A COs (CO1–CO4 are addressed substantively in this report; CO5–CO12 will be addressed in CSE400B once implementation begins). This explicit mapping helps the supervisor and examiner verify alignment with the EWU CSE400A guideline.

A.1 Knowledge Profile (K) Reference

K1 natural sciences; K2 mathematics, numerical analysis, statistics, formal computer science; K3 engineering fundamentals; K4 specialist engineering knowledge in computer science and engineering; K5 engineering design; K6 engineering practice (technology); K7 societal, health, safety, legal, cultural considerations; K8 engineering research literature.

A.2 CO → K → EP Mapping Table

CO	CO Description	Knowledge Profile (K) where addressed	Complex Engineering Problem (EP) where addressed
CO1	Integrate new and previously acquired knowledge for identifying a real-life complex engineering problem as the capstone project.	Background (Section 2): K1 (signal/image foundations for OCR and CV), K2 (statistics and ML for classification and anomaly scoring), K3 (system architecture, networking, browser extension model, software engineering).	Background (Section 2): EP1 in-depth ML, NLP, anomaly detection, and security knowledge required to define the problem. Research Questions (Section 4): EP6 conflicting requirements across end users, organizations, regulators, and privacy advocates.
CO2	Examine various problem domains (literature review), define the problems, and formulate the objectives for the capstone project.	Related Works (Section 3): K8 twenty recent research papers reviewed and synthesized into observations and gaps that motivate the proposed method.	Related Works (Section 3): EP1 and EP3 deep technical reading and synthesis. Objectives (Section 5): EP2, EP6, EP7 trade-offs between accuracy and latency, on-device vs cloud, detection power vs privacy; multiple stakeholders; and interdependent sub-problems (URL, OCR, NLP, anomaly, alerting).
CO3	Plan a methodology, system architecture, and data analysis approach appropriate for the identified problem.	Methodology (Section 6): K2, K3, K4, K5 selection of Random Forest, Isolation Forest, design of the three-layer architecture; specialist CS knowledge in OCR, NLP, and anomaly detection. Data Analysis Plan (Section 7): K2 and K8.	Methodology (Section 6): EP2 supervised vs unsupervised, on-device vs cloud; EP3 depth of analysis in module-level methodology and decision fusion; EP6 privacy vs detection power. Implementation Boundary (Section 6.8): EP2 and EP7 explicit partition of core, stretch, and future-work to manage interdependent sub-problems.
CO4	Apply societal, legal, ethical, and privacy	Security, Privacy, and Compliance (Section	Section 6.6 and Section 7.6: EP5 applicable codes and

CO	CO Description	Knowledge Profile (K) where addressed	Complex Engineering Problem (EP) where addressed
	considerations relevant to the engineering problem.	6.6): K7 GDPR Article 6(1)(a) consent principle and Bangladesh Digital Security Act expectations. Ethical and Privacy Considerations (Section 7.6): K7 informed consent for voice and screenshot data, data minimization, anonymization. Dataset Legal and Ethical Use (Section 7.1.1): K7 license terms (PhishTank, OpenPhish, Kaggle CC-BY), non-redistribution of brand pages.	standards (GDPR principles, Bangladesh DSA); EP6 conflicting requirements between detection power, user privacy, and regulatory expectations.
CO5	Investigate, evaluate, and select appropriate engineering tools and techniques to solve the identified problem.	To be addressed in CSE400B during implementation and evaluation. Preliminary tool selection is documented in Sections 6.1 and 6.7.	CSE400B EP3 and EP4.
CO6	Implement and integrate the proposed system as per the planned methodology.	To be addressed in CSE400B. The core deliverables are listed in Table 8.1.	CSE400B EP1, EP3, EP7.
CO7	Test and evaluate the implemented system against the defined objectives.	To be addressed in CSE400B. Evaluation metrics, targets, and analysis methods are defined in Sections 6.5 and 7.5.	CSE400B EP3.
CO8	Apply project management principles, including risk identification and mitigation.	Risk identification is partially addressed in this report: Section 6.8 (Implementation Boundary) and Section	CSE400A EP2 and EP7 (scope partition); CSE400B full lifecycle.

CO	CO Description	Knowledge Profile (K) where addressed	Complex Engineering Problem (EP) where addressed
		7.1.1 (Dataset Availability, Risk, and Fallback Plan). Full project management will be applied in CSE400B.	
CO9	Engage in lifelong learning by adapting to new tools, techniques, and standards.	Demonstrated by adopting recent (2023–2024) literature on Deep Isolation Forest [18], quantized LLMs [16], and multilingual phishing detection [11, 12], and by selecting modern frameworks (PyTorch, Hugging Face, FastAPI, Manifest V3) in Section 6.1.	CSE400A and CSE400B EP4 (familiarity with emerging issues).
CO10	Demonstrate professional and ethical responsibility in engineering practice.	Same as CO4 plus: honest scope partition (Section 6.8), honest dataset risk reporting (Section 7.1.1), and honest reference handling in Section 9 (placeholders are flagged rather than fabricated).	CSE400A EP5 and EP6.

Table A.1 : Mapping of CSE400A Course Outcomes to the Knowledge Profile (K) and Complex Engineering Problem (EP) attributes, with the report sections that address each one.